CODEMITVISHAL · FREE COURSE

Git Pro

Git from **zero to hero**: commits, branches, rebase, GitHub teamwork, internals, debugging, and the questions every interview asks.

12	12	100+	0 €
modules	hands-on exercises	interview answers	always free

COURSE MAP

From your first commit to rescuing any repository

#	MODULE	YOU'LL BE ABLE TO…
01	**Git basics**	why version control; install & config
02	**Your first repository**	add, commit, diff, log, .gitignore
03	**Undoing things**	restore, amend, reset, revert, reflog
04	**Branches & merging**	branches, fast-forward, conflicts
05	**Rebase & history**	rebase, rebase -i, cherry-pick
06	**Remotes & GitHub**	clone, push, pull, force-with-lease
07	**Team workflows**	GitHub flow, PRs, reviews, commits
08	**Stash, tags & releases**	stash, SemVer, release notes
09	**Git internals**	blobs, trees, commits, refs, HEAD
10	**Debugging with history**	log, blame, pickaxe, bisect
11	**Advanced Git**	hooks, worktrees, submodules, secrets
12	**Capstone**	rescue a messy repo, ship a release

HOW EVERY MODULE WORKS

Real Git, real output, real practice

1. **Why**: the real problem this solves
2. **Picture it**: an everyday analogy + a diagram
3. **Try it**: real Git commands with their **real output**
4. **Common mistake**: wrong vs right, side by side
5. **In real jobs**: how teams use it
6. **Interview**: what to say + the follow-up
7. **Recap** + a hands-on exercise

IN EVERY MODULE FOLDER

`Lessons`	run real Git in a safe sandbox
`Exercise`	setup → you fix it → check
`Quiz`	QUIZ.md with an answer key
`Interview`	INTERVIEW.md with model answers
`Slides`	slides/modules/Module-NN.pdf

HOW TO STUDY

Get the most out of Git Pro

1. Watch or read the module's **slides**.
2. **Read every lesson** and type each command yourself in Git Bash.
3. Do the **exercise** in its playground repo until **All checks passed!**
4. Take the **quiz**, then answer **INTERVIEW.md** out loud.
5. Before an interview: read **interview_prep/**.

```
# needs only Git (Git Bash on Windows)
git clone https://github.com/
    CodeMitVishal/git-pro.git
cd git-pro
bash exercises/ex01_first_repo/setup.sh
cd playground/ex01   # do the task
bash ../../exercises/ex01_first_repo/check.sh
```

MODULE

01

Git basics

What version control is, why every team uses Git, and how to set it up.

YOU WILL LEARN

Why version control	`01_why_version_control.md`
Install & configure	`02_install_and_config.md`
Exercise: first repository	`ex01_first_repo/`
Interview Q&A	`INTERVIEW.md`

WHY VERSION CONTROL

final_v2_REALLY_final.py

THE PROBLEM

Without version control, people copy files: **app_old.py, app_backup.py, app_final_v2.py**. Nobody knows which is current, what changed, or why, and two people editing the same file overwrite each other.

THE IDEA

Git saves snapshots of the whole project, called commits. Each one records what changed, who, when and why.

PICTURE IT VERSION CONTROL

A photo album of your project

📷 Every **commit** is a photo of the whole project with a caption (the message). You can flip back to any photo, compare two photos, and the team can take photos in parallel on **branches**.

`edit files`
your working folder

→

`git add`
choose what goes in the photo

→

`git commit`
take the photo + caption

→

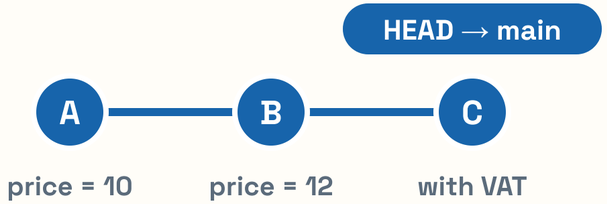
`git log`
flip through the album

Git is **distributed**: every clone has the whole album, so you can work offline.

PICTURE IT HISTORY

History = a chain of snapshots

Three commits on main



1. Each circle is a **commit**: a full snapshot.
2. Each commit points back to its **parent**.
3. **main** is a label on the newest commit; **HEAD** says where you are.

TRY IT VERSION CONTROL

Three snapshots of one file

```
$ git init
Initialized empty Git repository in ~/project/.git/
$ echo "price = 10" > shop.py
$ git add shop.py
$ git commit -m "Add shop with a fixed price"
[main (root-commit) f8b0491] Add shop with a fixed price
 1 file changed, 1 insertion(+)
 create mode 100644 shop.py
$ echo "price = 12" > shop.py
$ git commit -qam "Raise the price to 12"
$ git log --oneline
cacef8e Raise the price to 12
f8b0491 Add shop with a fixed price
$ git show HEAD~1:shop.py
price = 10
```

1. Each commit has an ID (a hash) like **f8b0491**.
2. **HEAD~1** = one commit before the current one.
3. Old versions are always there: nothing is lost.

STEP BY STEP GIT VS GITHUB

Git is not GitHub

	GIT	GITHUB / GITLAB / AZURE DEVOPS
What it is	a program on your computer	a website that hosts Git repositories
Works offline?	yes	no
Gives you	commits, branches, history, merges	pull requests, code review, issues, CI/CD
Command	`git commit`	`git push` sends commits there

STEP BY STEP SETUP

Configure Git once per machine

COMMAND	WHY
`git config --global user.name "Your Name"`	stored in every commit
`git config --global user.email "you@example.com"`	links commits to your GitHub account
`git config --global init.defaultBranch main`	new repos start on **main**
`git config --global core.editor "code --wait"`	VS Code for commit messages
`git config --local user.email "you@company.com"`	a different email in ONE repo

> Three levels: **--system** < **--global** (~/.gitconfig) < **--local** (.git/config). The most specific wins.

TRY IT SETUP

Local beats global

```
$ git init
Initialized empty Git repository in ~/project/.git/
$ git config --global user.email
vishal@example.com
$ git config --local user.email "vishal@company.com"
$ git config --show-scope --get-all user.email
global  vishal@example.com
local   vishal@company.com
$ git config user.email
vishal@company.com
```

1. The global value applies everywhere.
2. The local value wins inside this repository.
3. **--show-scope** shows where each value comes from.

COMMON MISTAKE SETUP

Committing with the wrong identity

✗ WRONG

```
# fresh laptop, nothing configured
git commit -m "Fix login"
# Author: acer <acer@LAPTOP-123.local>
```

Your commits show a random machine name and aren't linked to your GitHub profile, so your contributions don't count.

✓ RIGHT

```
git config --global user.name "Vishal"
git config --global user.email "vishal@example.com"
git commit --amend --reset-author --no-edit
```

Configure once per machine. Fix the last commit with **--amend --reset-author**.

IN REAL JOBS GIT

Git on your first day at work

Get the code

```
git clone git@github.com:company/shop.git
```

Set your work identity

```
git config --local user.email "you@company.com"
```

Daily loop

```
git add -p
git commit -m "Fix coupon rounding"
git push
```

Everything is reviewed

```
open a Pull Request -> review -> merge
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What is Git, and why do teams use it?”

- Git is a **distributed version control system**.
- It records snapshots (**commits**) with who, when and why, so you can compare and restore any version.
- Every clone has the **full history**, so you can work offline and every copy is a backup.
- **Branches** let many developers work in parallel and merge their work safely.

> **Follow-up they ask next:** ”Git vs GitHub?” (Git is the tool; GitHub hosts repositories and adds pull requests, reviews and CI).

RECAP REMEMBER THESE

Module 1 in 5 lines

1. A **commit** = a snapshot + who, when, why.
2. Git is **distributed**: every clone has the full history.
3. Git is the tool; GitHub hosts repositories.
4. Set **user.name** and **user.email** once per machine.
5. Config levels: system < global < local.

PRACTICE NOW

Run bash exercises/ex01_first_repo/setup.sh and make your first commit.

MODULE

02

Your first repository

The everyday loop: edit, add, commit. Read history and keep junk out.

YOU WILL LEARN

The three areas	01_three_areas.md
diff & log	02_diff_and_log.md
.gitignore	03_gitignore.md
Exercise: clean commits	ex02_clean_commits/
Interview Q&A	INTERVIEW.md

WHY THE STAGING AREA

Why add before commit?

THE PROBLEM

You fixed a bug, added debug prints and started a new feature, all in the same files. One big commit called **”stuff”** makes the history useless.

THE IDEA

The staging area lets you pick exactly what goes into each commit: one commit per idea.

PICTURE IT THE THREE AREAS

Desk, box, shelf

📦 Your **desk** is the working tree where you edit. **git add** puts things in an open **box** (the staging area). **git commit** seals the box with a label and puts it on the **shelf** (the repository).

working tree
files you edit

→

staging area (index)
the next commit, being packed

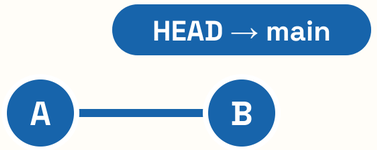
→

repository (.git)
sealed commits, forever

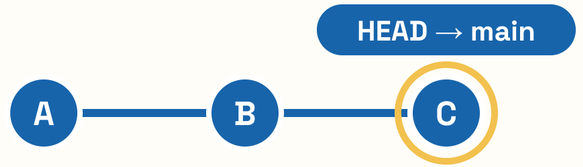
PICTURE IT COMMITS

Every commit adds one snapshot

Before




```
git commit -m "Add cart"
```



The new commit **C** points to its parent **B**, and the branch label **main** (with HEAD) moves forward to it.

TRY IT THE THREE AREAS

Follow one file through the three areas

```
$ git init
Initialized empty Git repository in ~/project/.git/
$ echo "print('hello')" > app.py
$ git status --short
?? app.py
$ git add app.py
$ git status --short
A  app.py
$ git commit -m "Add hello app"
[main (root-commit) 5798825] Add hello app
 1 file changed, 1 insertion(+)
 create mode 100644 app.py
$ git status --short
```

1. **??** untracked: Git sees it, doesn't save it.
2. **A** = added to the staging area.
3. After the commit, the status is clean.

TRY IT PARTIAL COMMITS

Commit only what belongs together

```
$ echo "print('hello, world')" > app.py
$ echo "debug notes" > notes.txt
$ git status --short
 M app.py
?? notes.txt
$ git add app.py
$ git status --short
M  app.py
?? notes.txt
$ git commit -m "Greet the whole world"
[main e49f271] Greet the whole world
 1 file changed, 1 insertion(+), 1 deletion(-)
```

1. Left column = staged, right column = not staged.
2. **notes.txt** stays out of the commit.
3. Interactive picking inside a file: **git add -p**.

TRY IT DIFF

What exactly changed?

```
$ echo "pears = 5" >> cart.py
$ git diff
diff --git a/cart.py b/cart.py
index c8fcf4f..881c683 100644
--- a/cart.py
+++ b/cart.py
@@ -1 +1,2 @@
 apples = 3
+pears = 5
$ git add cart.py
$ git diff --staged --stat
 cart.py | 1 +
 1 file changed, 1 insertion(+)
```

1. **git diff**: changes NOT yet staged.
2. **git diff --staged**: what the next commit will contain.
3. Read your diff before every commit.

STEP BY STEP HISTORY

Reading the log

COMMAND	SHOWS
`git log --oneline`	one line per commit
`git log --stat`	which files changed in each commit
`git log -p -- cart.py`	every change to one file
`git show HEAD`	the last commit: message + diff
`git log --oneline --graph --all`	branches as a picture (Module 4)

TRY IT .GITIGNORE

Keep junk and secrets out

```
$ git status --short --untracked-files=all
?? .env
?? __pycache__/app.pyc
?? app.py
?? server.log
$ echo ".env" > .gitignore
$ echo "*.log" >> .gitignore
$ echo "__pycache__/" >> .gitignore
$ git status --short --untracked-files=all
?? .gitignore
?? app.py
$ git check-ignore -v server.log
.gitignore:2:*.log      server.log
```

1. Patterns: names, ***.log**, folders with **/**.
2. **check-ignore -v** tells you which rule matched.
3. Templates: github.com/github/gitignore.

COMMON MISTAKE .GITIGNORE

Ignoring a file that is already tracked

✗ WRONG

```
git add settings.ini
git commit -m "Add app"
echo "settings.ini" >> .gitignore
# settings.ini is STILL tracked
```

.gitignore only affects **untracked** files. Once committed, a file stays tracked.

✓ RIGHT

```
git rm --cached settings.ini
git commit -m "Stop tracking settings.ini"
# the file stays on disk, Git forgets it
```

git rm --cached removes it from the index only. A leaked secret is still in history: rotate it.

IN REAL JOBS EVERYDAY GIT

The loop you'll run 20 times a day

See where you are

```
git status -sb
```

Review before committing

```
git diff
git add -p
```

Commit one idea

```
git commit -m "Fix rounding in coupon total"
```

Check the result

```
git log --oneline -5
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What is the staging area, and why does it exist?”

- The staging area (index) is the **snapshot being prepared** for the next commit.
- **git add** copies changes into it; **git commit** saves it as a commit.
- It lets you commit **part** of your work: focused commits that are easy to review and revert.
- **git diff** shows unstaged changes; **git diff --staged** shows staged ones.

> **Follow-up they ask next:** ”A file is in .gitignore but still tracked: why?” (it was committed before; use git rm --cached).

RECAP REMEMBER THESE

Module 2 in 5 lines

1. Working tree → **git add** → staging area → **git commit** → repository.
2. **git status -s**: ?? untracked, A added, M modified.
3. **git diff** vs **git diff --staged**.
4. **git log --oneline**, **--stat**, **-p**, **git show**.
5. **.gitignore** for junk and secrets; **git rm --cached** for tracked files.

PRACTICE NOW

Do exercises/ex02_clean_commits/: three focused commits and a .gitignore.

MODULE

03

Undoing things

Every mistake has an undo: restore, amend, reset, revert, and the reflog safety net.

YOU WILL LEARN

restore & amend	01_restore_and_amend.md
reset: soft, mixed, hard	02_reset_three_modes.md
revert & reflog	03_revert_and_reflog.md
Exercise: undo three mistakes	ex03_undo_mistakes/
Interview Q&A	INTERVIEW.md

WHY UNDO

Mistakes are normal. Panic is optional.

THE PROBLEM

A broken experiment in a file, a typo in a commit message, three commits that should have been one, a bad commit already on main, a branch deleted by accident...

THE IDEA

Pick the undo by asking two questions: where is the mistake (file, staging, commit)? And is it already shared?

STEP BY STEP UNDO

Which undo do I need?

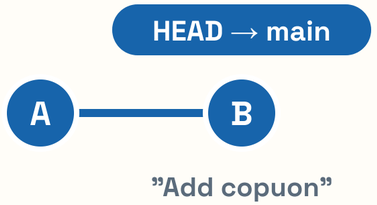
SITUATION	COMMAND
Throw away edits in a file	`git restore app.py`
Unstage a file (keep the edit)	`git restore --staged app.py`
Fix the last commit (not pushed)	`git commit --amend`
Undo local commits	`git reset --soft | --mixed | --hard HEAD~N`
Undo a commit that's already **shared**	`git revert <commit>`
"I lost commits!"	`git reflog` → reset / branch to it

> Shared history (pushed main) is never rewritten: **revert** it.

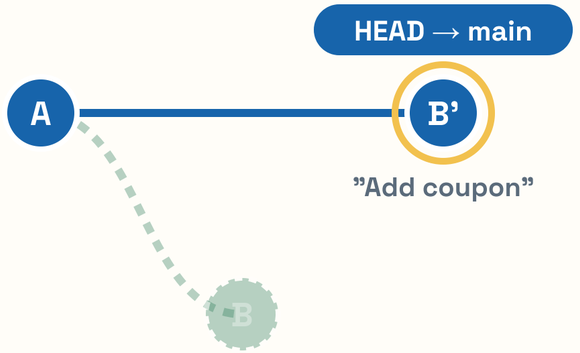
PICTURE IT AMEND

amend replaces the last commit

Before: typo in B



`git commit --amend`



B' is a brand-new commit (new ID). The old **B** is abandoned, so never amend what others already pulled.

TRY IT RESTORE AND AMEND

Fix the last commit

```
$ git add coupon.py
$ git commit -m "Add copuon"
[main 6e010f1] Add copuon
 1 file changed, 1 insertion(+)
 create mode 100644 coupon.py
$ git add test_coupon.py
$ git commit --amend -m "Add coupon with a test"
[main 1033dbe] Add coupon with a test
 Date: Tue Oct 27 08:05:00 2026 +0000
 2 files changed, 2 insertions(+)
 create mode 100644 coupon.py
 create mode 100644 test_coupon.py
$ git log --oneline --stat -1
1033dbe Add coupon with a test
 coupon.py      | 1 +
 test_coupon.py | 1 +
 2 files changed, 2 insertions(+)
```

1. Typo and a forgotten file, fixed in one step.
2. **--no-edit** keeps the old message.
3. The ID changes: only amend **unpushed** commits.

PICTURE IT RESET

reset moves the branch back

🔖 Like moving a **bookmark** back in a book. What happens to the pages after it? **--soft**: a neat pile on your desk (staged). **--mixed**: loose on the desk (unstaged). **--hard**: the shredder.

`--soft`
branch moves, changes STAGED

→

`--mixed (default)`
branch moves, changes UNSTAGED

→

`--hard`
branch moves, changes DELETED

TRY IT RESET

Same reset, three modes

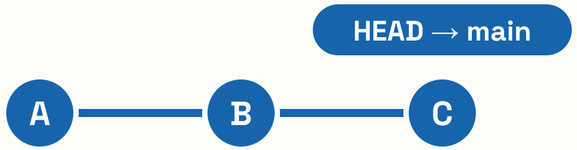
```
$ git reset --soft HEAD~2
$ git status --short
M  app.txt
$ git reset --mixed HEAD
Unstaged changes after reset:
M       app.txt
$ git status --short
 M app.txt
$ git reset --hard HEAD
HEAD is now at bcf013a Version 1
$ git status --short
$ cat app.txt
v1
```

1. After **--soft**: M in the LEFT column (staged).
2. After **--mixed**: M in the RIGHT column (unstaged).
3. After **--hard**: clean, the v3 content is gone.

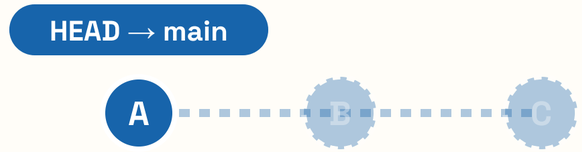
PICTURE IT RESET

reset moves the branch label back

Before



`git reset HEAD~2`



B and C are no longer on main (reflog still finds them). **--soft**, **--mixed**, **--hard** only decide what happens to their **changes**.

TRY IT REVERT

Undo a shared commit safely

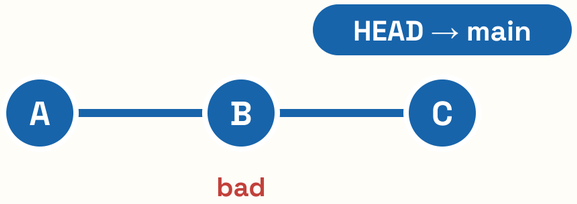
```
$ git log --oneline
62d2edc Add tax
dbce891 Change pricing
29988e9 Add shop
$ git revert --no-edit HEAD~1
[main fb0eb65] Revert "Change pricing"
 Date: Tue Oct 27 08:08:00 2026 +0000
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline
fb0eb65 Revert "Change pricing"
62d2edc Add tax
dbce891 Change pricing
29988e9 Add shop
$ cat shop.txt
price = 10
```

1. **revert** adds a NEW commit that does the opposite.
2. History stays intact: safe for main.
3. Undo a merge: **git revert -m 1 <merge>**.

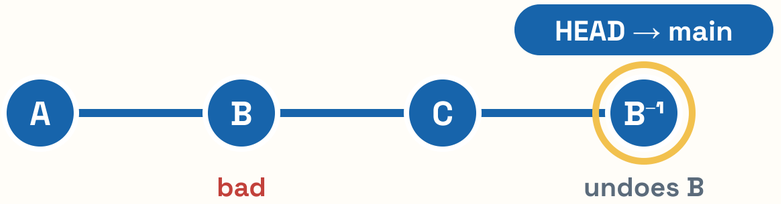
PICTURE IT REVERT

revert adds an ”undo” commit

B broke production



`git revert B`



Nothing is rewritten: history only moves **forward**. That’s why revert is safe on main.

TRY IT REFLOG

The safety net: nothing committed is lost

```
$ git reset --hard HEAD~2
HEAD is now at bcf013a Version 1
$ git log --oneline
bcf013a Version 1
$ git reflog -3
bcf013a HEAD@{0}: reset: moving to HEAD~2
1d36b1f HEAD@{1}: commit: Version 3
49ea172 HEAD@{2}: commit: Version 2
$ git reset --hard HEAD@{1}
HEAD is now at 1d36b1f Version 3
$ git log --oneline
1d36b1f Version 3
49ea172 Version 2
bcf013a Version 1
```

1. **reflog** = every place HEAD has been.
2. Find the commit, then reset or **git branch rescue <id>**.
3. Kept ~90 days, local only.

COMMON MISTAKE SHARED HISTORY

Rewriting history that others have

✗ WRONG

```
git reset --hard HEAD~1     # on main
git push --force
# teammates' next pull: chaos
```

Everyone who pulled the old commit now has history that conflicts with yours. Work gets lost or duplicated.

✓ RIGHT

```
git revert HEAD
git push
# history only moves forward
```

Rule: **reset/amend/rebase** only local commits; **revert** shared ones.

IN REAL JOBS UNDO

Undo at work

Hotfix: undo yesterday's release commit

```
git revert a1b2c3d
git push    # CI deploys the fix
```

Clean up before a pull request

```
git reset --soft origin/main
git commit -m "Add CSV export"
```

Committed to main by mistake

```
git branch feature/csv
git reset --hard origin/main
```

"I lost my work!"

```
git reflog
git branch rescue HEAD@{3}
```

INTERVIEW SAY THIS IN THE INTERVIEW

“git reset vs git revert?”

- **reset** moves the branch pointer back: history is rewritten. Use it only for **local** commits.
- **revert** creates a **new** commit that undoes an old one: history is kept, so it’s safe on shared branches.
- reset modes: **--soft** keeps changes staged, **--mixed** unstaged, **--hard** discards them.
- If you lose commits: **git reflog**.

> **Follow-up they ask next:** ”How do you undo a pushed merge?” (git revert -m 1 <merge-commit>).

RECAP REMEMBER THESE

Module 3 in 5 lines

1. **restore** files, **restore --staged** to unstage.
2. **commit --amend** fixes the last unpushed commit.
3. **reset --soft/--mixed/--hard** moves the branch back.
4. **revert** for anything already shared.
5. **reflog** finds ”lost” commits.

PRACTICE NOW

Do exercises/ex03_undo_mistakes/: three mistakes, three undos.

MODULE

04

Branches & merging

Build features in parallel without breaking main, then bring the work together.

YOU WILL LEARN

Branches are pointers	`01_branches_are_pointers.md`
Fast-forward vs three-way	`02_fast_forward_and_three_way.md`
Merge conflicts	`03_merge_conflicts.md`
Exercise: resolve a conflict	`ex04_merge_conflict/`
Interview Q&A	`INTERVIEW.md`

WHY BRANCHES

Why branches?

THE PROBLEM

You’re halfway through a big feature when a production bug comes in. Your half-done code can’t be released, and five teammates are changing the same project at the same time.

THE IDEA

A branch is a separate line of work. main stays releasable; each feature or fix lives on its own branch until it’s merged.

PICTURE IT BRANCHES

A branch is a sticky note on a commit

📌 Commits form a timeline. A branch is just a **sticky note** with a commit ID on it; creating one copies nothing. **HEAD** is the note you're holding: new commits move that note forward.

`main`
points to the release commit

→

`feature/search`
points to your newest commit

→

`HEAD`
points to the branch you're on

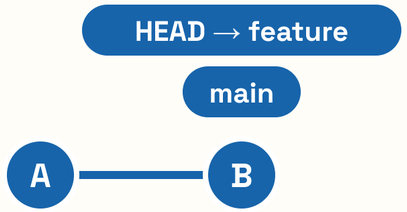
→

`new commit`
moves your branch forward

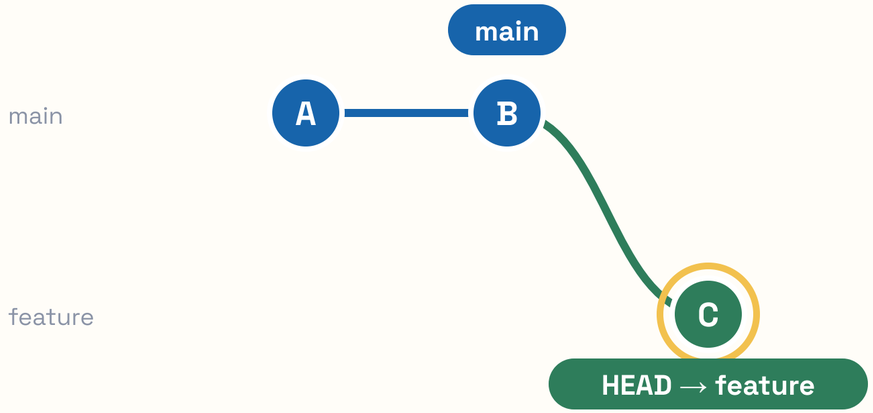
PICTURE IT BRANCHES

Creating a branch copies nothing

git switch -c feature



git commit (on feature)



A branch is just a label. Only the branch HEAD points to moves when you commit.

TRY IT BRANCHES

Create, commit, switch

```
$ git switch -c feature/coupons
Switched to a new branch 'feature/coupons'
$ echo "coupons" > coupons.txt
$ git add coupons.txt
$ git commit -m "Add coupons"
[feature/coupons 93b1d0e] Add coupons
 1 file changed, 1 insertion(+)
 create mode 100644 coupons.txt
$ git log --oneline --all --graph
* 93b1d0e Add coupons
* cbae48b Add home page
$ git switch main
Switched to branch 'main'
$ git ls-files
home.txt
```

1. **switch -c**: create + switch in one step.
2. main doesn't know about coupons.txt yet.
3. Switching branches changes the files in your folder.

TRY IT MERGING

Fast-forward: main didn’t move

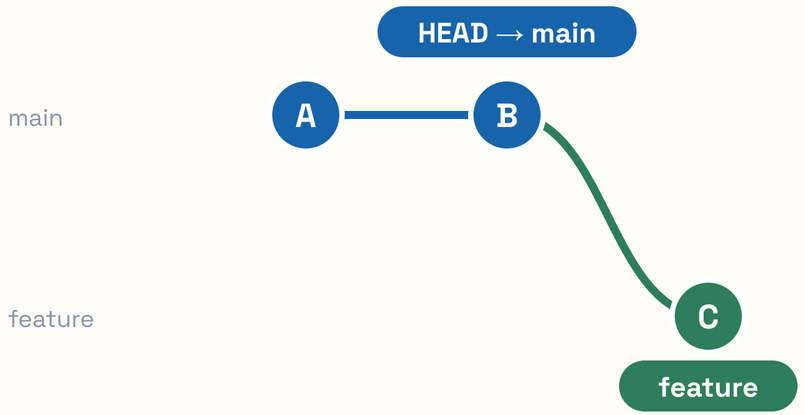
```
$ git switch main
Switched to branch 'main'
$ git merge feature/search
Updating cbae48b..bed5ce2
Fast-forward
 search.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 search.txt
$ git log --oneline --graph --all
* bed5ce2 Add search
* cbae48b Add home page
```

1. Git just slides main forward.
2. No merge commit: history stays a straight line.

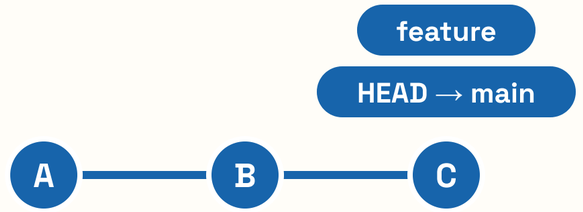
PICTURE IT FAST-FORWARD

Fast-forward merge

Before



`git merge feature`



main had no new commits, so Git simply **slides the label forward**. No merge commit.

TRY IT MERGING

Three-way: both branches moved

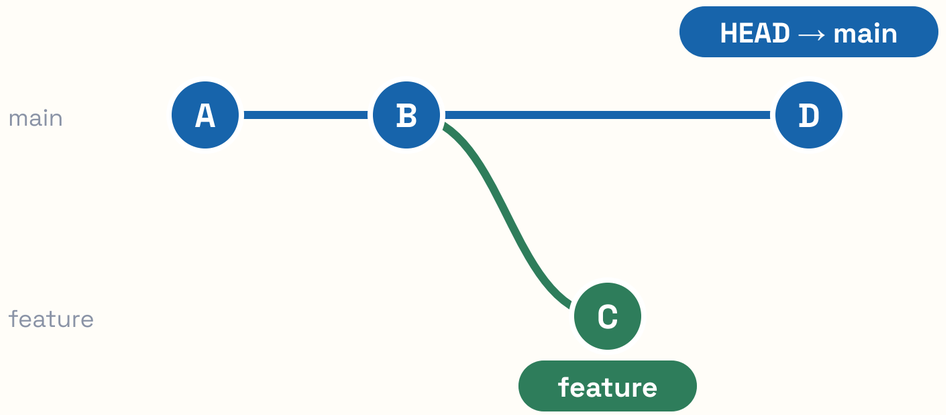
```
$ git switch main
Switched to branch 'main'
$ echo "footer" > footer.txt
$ git add footer.txt
$ git commit -qm "Add footer"
$ git merge --no-edit feature/search
Merge made by the 'ort' strategy.
 search.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 search.txt
$ git log --oneline --graph --all
*   28c942c Merge branch 'feature/search'
|\
| * bed5ce2 Add search
* | 9e81205 Add footer
|/
* cbae48b Add home page
```

1. Git creates a **merge commit** with two parents.
2. The graph shows the two lines joining.
3. **--no-ff** forces a merge commit even when a fast-forward is possible.

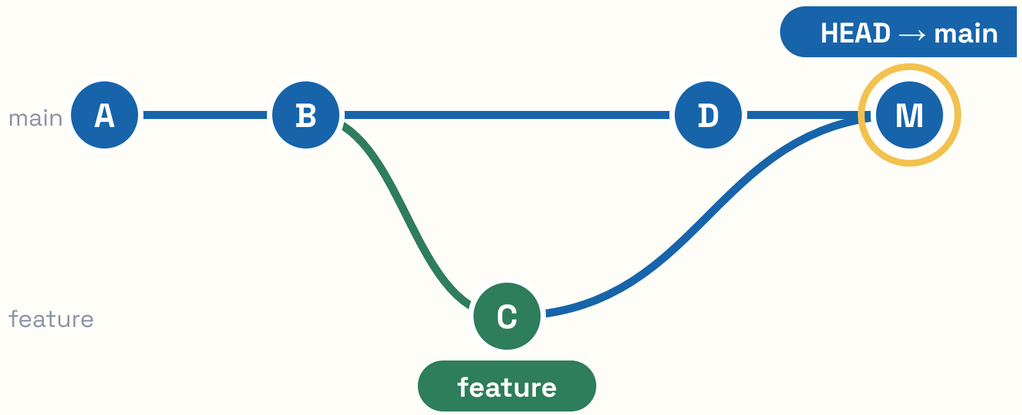
PICTURE IT THREE-WAY MERGE

Three-way merge

Before



`git merge feature`



Both sides have new work, so Git creates **M**, a merge commit with **two parents**.

TRY IT CONFLICTS

Both branches changed the same line

```
$ git merge feature/sale
Auto-merging banner.txt
CONFLICT (content): Merge conflict in banner.txt
Automatic merge failed; fix conflicts and then commit the result.
$ git status --short
UU banner.txt
$ cat banner.txt
<<<<<<< HEAD
Welcome! Free shipping today
=======
Big SALE: 20% off
>>>>>>> feature/sale
```

1. **UU** = unmerged: Git needs you.
2. Between **<<<<<<<** and **=======**: your branch (HEAD).
3. Between **=======** and **>>>>>>>**: theirs.

STEP BY STEP CONFLICTS

Resolving a conflict in 4 steps

STEP	DO
1 Find	`git status` : every UU file
2 Decide	edit the file to the final content (keep one side, the other, or a mix); delete the markers
3 Mark resolved	`git add banner.txt`
4 Finish	`git commit` (run the tests first!)
or cancel	`git merge --abort`

> VS Code shows buttons: Accept Current / Incoming / Both. Still read the result.

COMMON MISTAKE CONFLICTS

Committing conflict markers

✗ WRONG

```
git add .
git commit -m "Merge sale"
# banner.txt now contains:
# <<<<<<< HEAD ... ======= ... >>>>>>>
```

The site shows raw markers, or the code doesn't even compile. It happens when you **git add .** without looking.

✓ RIGHT

```
git diff --check        # finds leftover markers
git grep -n "<<<<<<<"
# fix, test, then add and commit
```

Check for markers and run the tests before committing a resolution.

IN REAL JOBS BRANCHES

Branching at work

One branch per ticket

```
git switch -c feature/SHOP-142-coupons
```

Keep up with main

```
git fetch
git merge origin/main      # or rebase (Module 5)
```

Merge through a pull request

```
push the branch -> open PR -> review -> merge
```

Tidy up

```
git branch --merged main
git branch -d feature/SHOP-142-coupons
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you resolve a merge conflict?”

- Conflicts happen when both branches changed the **same lines**.
- **git status** lists the conflicted files; open each one and decide the final content.
- Remove the **<<<<<<< ======= >>>>>>>** markers, **git add**, then **git commit**.
- Run the tests; **git merge --abort** cancels everything.
- Avoid them: small branches, merge main often.

> **Follow-up they ask next:** ”Fast-forward vs three-way merge?” (no new commits on the target: just move the pointer; otherwise a merge commit with two parents).

RECAP REMEMBER THESE

Module 4 in 5 lines

1. A branch = a movable pointer; HEAD = where you are.
2. **git switch -c** to create, **git switch** to move.
3. Fast-forward: just move the pointer. Three-way: a merge commit.
4. Conflicts: edit, remove markers, **add**, **commit** (or **--abort**).
5. Delete merged branches with **git branch -d**.

PRACTICE NOW

Do exercises/ex04_merge_conflict/: merge dark mode and resolve the conflict.

MODULE

05

Rebase & rewriting history

Clean, linear history: rebase, squash, reword and cherry-pick, plus the golden rule.

YOU WILL LEARN

rebase basics	01_rebase_basics.md
Interactive rebase	02_interactive_rebase.md
cherry-pick	03_cherry_pick.md
Exercise: clean up a branch	ex05_clean_history/
Interview Q&A	INTERVIEW.md

WHY REBASE

Why rewrite history?

THE PROBLEM

Your branch is 3 days behind main and has commits called **”wip”**, **”fix”**, **”fix again”**. Merging it adds noise and a tangled graph that’s hard to review, bisect or revert.

THE IDEA

Before sharing, rebase onto the latest main and squash the noise: reviewers see a few clear commits in a straight line.

PICTURE IT REBASE

Re-planting your branch

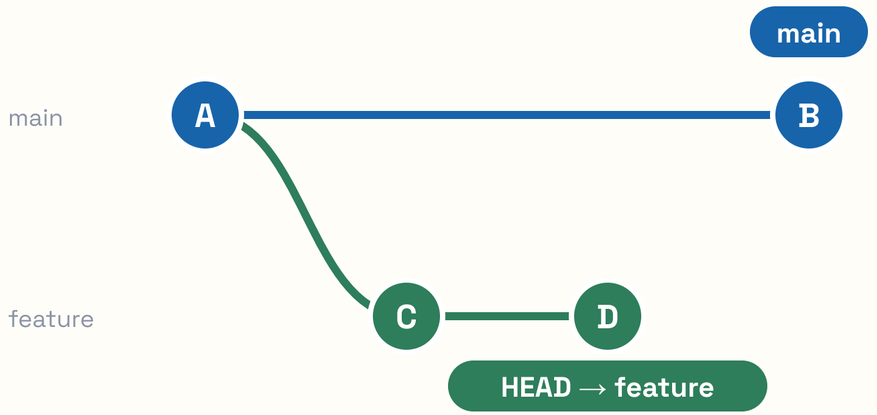
🌱 **merge** ties two branches together with a knot (a merge commit). **rebase** cuts your commits off and **re-plants** them on the newest tip of main: same changes, new commits, straight line.



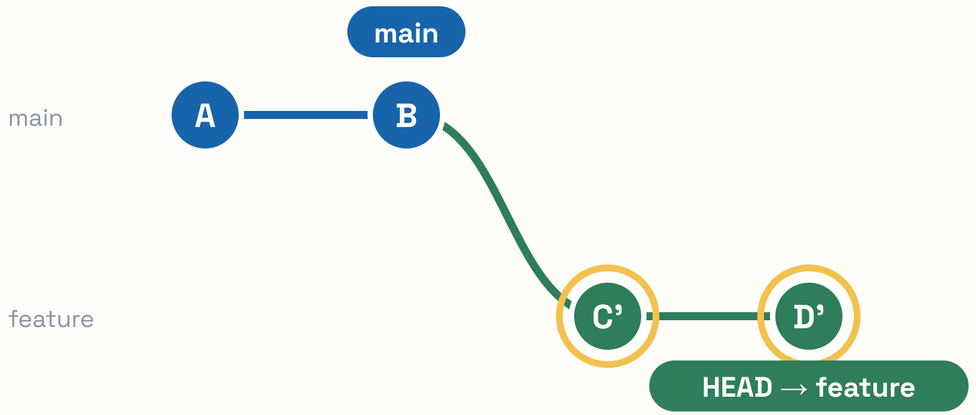
PICTURE IT REBASE

rebase replays your commits

Before



git rebase main



C and D are replayed on top of B as **new** commits C' and D'. Now main can fast-forward.

TRY IT REBASE

Replay the feature on the latest main

```
$ git log --oneline --graph --all
* ef6f46b Add footer
| * 572f72a Add search results
| * 92be44f Add search box
|/
* 4397492 Add home page
$ git switch -q feature/search
$ git rebase main
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feature/search.
$ git log --oneline --graph --all
* c5e801a Add search results
* c64e63a Add search box
* ef6f46b Add footer
* 4397492 Add home page
```

1. Before: two lines of history.
2. After: one straight line.
3. The feature commits have **new IDs**.

STEP BY STEP REBASE

merge vs rebase

	MERGE	REBASE
History	true history, with merge commits	linear, as if you started later
Rewrites commits?	no	**yes**: new IDs
Safe on shared branches?	yes	**no**: only your own commits
Conflicts	once, at the merge	possibly once per replayed commit
Typical use	merging a PR into main	updating your feature branch before a PR

> **Golden rule**: never rebase commits that other people already pulled.

TRY IT INTERACTIVE REBASE

git rebase -i: edit your history like a draft

```
# git rebase -i HEAD~3   opens this todo list:
pick  8510441 Add search
fixup 0a937f5 WIP
fixup 4f5af1c fix typo

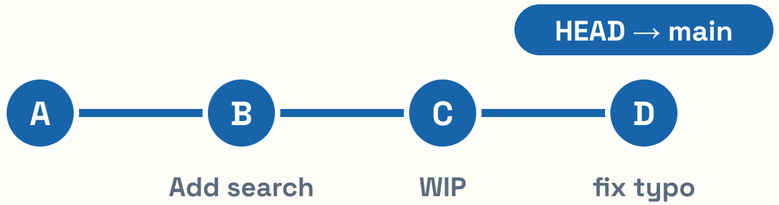
# pick   = keep          reword = new message
# squash = melt + keep the message
# fixup  = melt + drop the message
# drop   = delete        (reorder lines to reorder commits)
```

1. Edit the words, save, close: Git rewrites the commits.
2. Oldest commit at the TOP.
3. Abort any time: **git rebase --abort**.

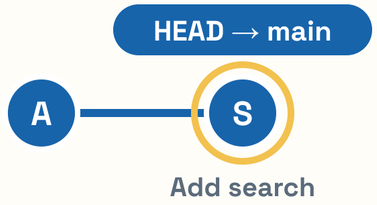
PICTURE IT INTERACTIVE REBASE

Squash: many commits become one

Before



`git rebase -i HEAD~3 (fixup)`



Reviewers see one clear commit instead of three noisy ones.

TRY IT INTERACTIVE REBASE

Three commits become one

```
$ git log --oneline
4f5af1c fix typo
0a937f5 WIP
8510441 Add search
04f6802 Add shop
$ git rebase -i HEAD~3
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/main.
$ git log --oneline
2799796 Add search
04f6802 Add shop
```

1. Here a tiny script plays you editing the todo list.
2. fixup + fixup = one clean **Add search** commit.
3. Shortcut: **git commit --fixup X** + **git rebase -i --autosquash**.

TRY IT CHERRY-PICK

Copy ONE commit to another branch

```
$ git log --oneline --all
3bbabdc Fix rounding in cart total
d69c0f5 Add new checkout
8f651f1 Add cart
$ git switch -q release/1.0
$ git cherry-pick -x main
[release/1.0 e4a7265] Fix rounding in cart total
 Date: Tue Oct 27 08:07:00 2026 +0000
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline
e4a7265 Fix rounding in cart total
8f651f1 Add cart
$ git ls-files
cart.py
```

1. Only the fix comes over, not the unreleased feature.
2. **-x** records the original commit ID.
3. Typical: backport a hotfix to a release branch.

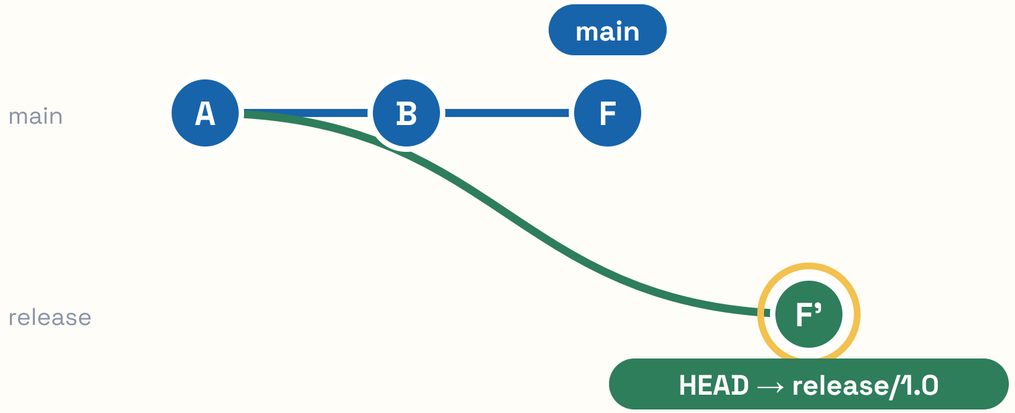
PICTURE IT CHERRY-PICK

cherry-pick copies one commit

Before



`git cherry-pick F`



Only the fix lands on the release branch, as a **copy** F' (new ID).

COMMON MISTAKE THE GOLDEN RULE

Rebasing a shared branch

✗ WRONG

```
git switch main
git rebase feature/x
git push --force
# everyone who pulled main is now broken
```

Rebase creates new commits. Teammates still have the old ones, and their next pull duplicates or loses work.

✓ RIGHT

```
git switch feature/x       # YOUR branch
git rebase origin/main
git push --force-with-lease
# main is only ever merged into
```

Rebase your **own** branch onto main; never rebase main or any shared branch.

IN REAL JOBS REBASE

Rebase at work

Update your branch before a PR

```
git fetch
git rebase origin/main
```

Address review comments neatly

```
git commit --fixup a1b2c3d
git rebase -i --autosquash origin/main
```

Avoid merge noise when pulling

```
git config --global pull.rebase true
```

Backport a fix

```
git switch release/2.3
git cherry-pick -x a1b2c3d
```

INTERVIEW SAY THIS IN THE INTERVIEW

“merge vs rebase: which do you use, and when?”

- **merge** preserves the true history with a merge commit: safe for shared branches.
- **rebase** replays my commits on a new base for a **linear** history, but rewrites them (new IDs).
- I rebase **my own** feature branch onto main before a PR, and merge or squash-merge the PR into main.
- Golden rule: never rebase commits others have pulled.

> **Follow-up they ask next:** ”How do you squash commits?” (git rebase -i with squash/fixup, or git reset --soft base + one commit, or Squash and merge on GitHub).

RECAP REMEMBER THESE

Module 5 in 5 lines

1. **rebase** = replay your commits on a new base: linear history.
2. Conflict while rebasing: fix, **add**, **rebase --continue**.
3. **rebase -i**: pick, reword, squash, fixup, drop, reorder.
4. **cherry-pick** copies single commits (hotfix backports).
5. Never rebase shared history.

PRACTICE NOW

Do exercises/ex05_clean_history/: squash and rebase a messy branch.

MODULE

06

Remotes & GitHub

Share work with a team: clone, push, pull, fetch, and force-push safely.

YOU WILL LEARN

clone, push, pull	01_clone_push_pull.md
fetch vs pull	02_fetch_vs_pull.md
Rejected pushes & force	03_rejected_push_and_force.md
Exercise: sync with your team	ex06_sync_with_team/
Interview Q&A	INTERVIEW.md

WHY REMOTES

Your laptop is not a team

THE PROBLEM

Commits on your laptop help nobody: the team can’t review them, CI can’t test them, and if the disk dies they’re gone.

THE IDEA

A remote (GitHub, GitLab, Azure DevOps) is the shared copy. You push your commits to it and pull everyone else’s.

PICTURE IT REMOTES

A shared bookshelf

📚 Everyone has a full copy at home. You **push** new pages to the shelf and **pull** pages others added. **origin/main** is your note of what the shelf looked like the last time you checked.

`git clone`
copy the whole shelf home

→

`git push`
put your new pages on the shelf

→

`git fetch`
check the shelf, update origin/*

→

`git pull`
fetch + bring the pages into your branch

TRY IT REMOTES

Connect a project and push it

```
$ git init --bare ../github/shop.git
Initialized empty Git repository in ~/github/shop.git/
$ git remote add origin ../github/shop.git
$ git remote -v
origin  ../github/shop.git (fetch)
origin  ../github/shop.git (push)
$ git push -u origin main
branch 'main' set up to track 'origin/main'.
To ../github/shop.git
 * [new branch]      main -> main
$ git status -sb
## main...origin/main
```

1. A bare repo plays GitHub (offline).
2. **-u** links main to origin/main for later pushes/pulls.
3. Real URL: **git@github.com:you/shop.git**.

TRY IT FETCH VS PULL

fetch first, look, then integrate

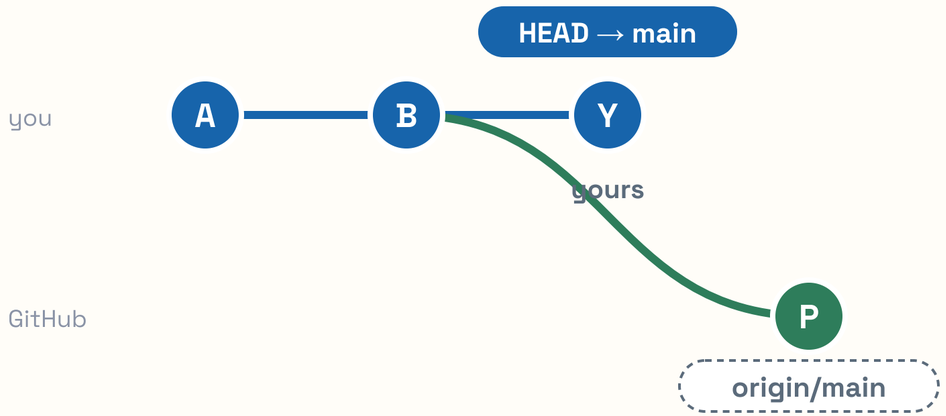
```
$ git fetch
From ../github/shop
   7cb5715..a04f8da  main       -> origin/main
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git log --oneline main..origin/main
a04f8da Add payments
$ git pull --rebase
Rebasing (1/1)
Successfully rebased and updated refs/heads/main.
$ git log --oneline --graph
* b16c859 Add shipping
* a04f8da Add payments
* 7cb5715 Initial commit
$ git push
To ../github/shop.git
   a04f8da..b16c859  main -> main
```

1. **fetch** moves only origin/main.
2. **ahead 1, behind 1**: both sides have new work.
3. **pull --rebase**: linear, no merge commit.

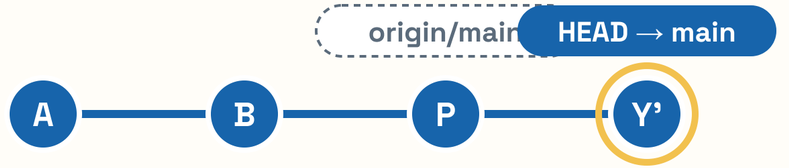
PICTURE IT REMOTES

ahead 1, behind 1

After git fetch



`git pull --rebase`



Your commit is replayed on top of Priya's. Now **git push** works and history stays a straight line.

STEP BY STEP REMOTES

Remote commands cheat sheet

COMMAND	DOES
`git clone <url>`	copy a repository with all history
`git remote -v` / `add` / `remove`	manage remote names and URLs
`git fetch --prune`	download new commits, drop deleted remote branches
`git pull --rebase`	fetch + rebase your commits on top
`git push -u origin feature/x`	publish a new branch and track it
`git push origin --delete feature/x`	delete a remote branch

TRY IT REJECTED PUSH

When the remote has work you don’t

```
$ git push
To ../github/shop.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to '../github/shop.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
$ git pull --rebase
From ../github/shop
   7cb5715..a04f8da  main       -> origin/main
Rebasing (1/1)
Successfully rebased and updated refs/heads/main.
$ git push
To ../github/shop.git
   a04f8da..c989af9  main -> main
```

1. Git refuses to overwrite Priya’s commit.
2. Integrate first, then push.
3. Never ”fix” this with --force on a shared branch.

COMMON MISTAKE FORCE PUSH

--force on a shared branch

✗ WRONG

```
git rebase -i HEAD~3      # on main
git push --force
# Priya's pushed commits: gone from GitHub
```

--force overwrites the remote no matter what's there. Teammates' work disappears from the server.

✓ RIGHT

```
git switch feature/cart     # your own branch
git rebase origin/main
git push --force-with-lease
# refuses if someone pushed meanwhile
```

Only force-push your own branch, and always with **--force-with-lease**. Protect main on GitHub.

IN REAL JOBS REMOTES

Remotes at work

Clone with SSH

```
ssh-keygen -t ed25519 -C "you@company.com"
git clone git@github.com:company/shop.git
```

Publish your branch

```
git push -u origin feature/SHOP-142
```

Contribute via a fork

```
git remote add upstream git@github.com:org/proj.git
git fetch upstream && git rebase upstream/main
```

Keep things tidy

```
git fetch --prune
git branch -vv
```

INTERVIEW SAY THIS IN THE INTERVIEW

“git fetch vs git pull?”

- **fetch** downloads new commits and updates remote-tracking branches like **origin/main**; my branches and files don’t change.
- **pull** = fetch + merge (or rebase with **--rebase**) into my current branch.
- I fetch first and look (**git log main..origin/main**), then integrate.
- A rejected push means the remote has work I don’t have: pull --rebase, then push.

> **Follow-up they ask next:** ”--force vs --force-with-lease?” (with-lease refuses to overwrite commits you haven’t seen; never force shared branches).

RECAP REMEMBER THESE

Module 6 in 5 lines

1. A remote = a shared copy; **origin** = the one you cloned.
2. **push -u** once, then **push** / **pull**.
3. **fetch** updates origin/*; **pull** = fetch + merge/rebase.
4. Rejected push → **pull --rebase** → push.
5. Force only your own branch, with **--force-with-lease**.

PRACTICE NOW

Do exercises/ex06_sync_with_team/: get Priya's commit and push yours.

MODULE

07

Team workflows

How real teams ship: GitHub flow, pull requests, reviews, commit conventions and branching strategies.

YOU WILL LEARN

GitHub flow	`01_github_flow.md`
Conventional Commits	`02_conventional_commits.md`
Branching strategies	`03_branching_strategies.md`
Pull requests & reviews	`04_pull_requests_and_reviews.md`
Exercise: great commits	`ex07_conventional_commits/`
Interview Q&A	`INTERVIEW.md`

WHY WORKFLOWS

Ten developers, one main branch

THE PROBLEM

Without rules, people push half-finished work to main, production breaks on Friday evening, and nobody knows which change caused it.

THE IDEA

A workflow is the team's agreement: branch per task, pull request with CI and review, then merge into a protected main.

PICTURE IT GITHUB FLOW

The GitHub flow loop

🍳 A restaurant kitchen: each dish (branch) is prepared at its own station and only reaches the customer (main) after the head chef tastes it (review + CI).



TRY IT MERGE BUTTONS

”Squash and merge” = one commit per PR

```
$ git merge --squash feature/wishlist
Squash commit -- not updating HEAD
Automatic merge went well; stopped before committing as requested
$ git commit -qm "Add wishlist (#42)"
$ git log --oneline --graph
* 878cf6e Add wishlist (#42)
* 55c1d7e Add footer
* 3d365c0 Initial commit
```

1. The 2 branch commits become 1 on main.
2. Easy to read; easy to revert a whole PR.
3. ”Create a merge commit” = **--no-ff**; ”Rebase and merge” = rebase + fast-forward.

STEP BY STEP MERGE BUTTONS

The three GitHub merge buttons

BUTTON	HISTORY ON MAIN	CHOOSE WHEN
`Create a merge commit`	all commits + a merge commit	you want the full branch story
`Squash and merge`	one commit per PR	most teams: clean main, easy reverts
`Rebase and merge`	linear, every commit kept	commits are already clean and meaningful

TRY IT CONVENTIONAL COMMITS

Commit messages that tools can read

```
feat(cart): add coupon codes          -> minor  1.2.0 -> 1.3.0
fix(cart): round totals to 2 decimals -> patch  1.2.0 -> 1.2.1
docs: explain how to run the shop
refactor(search): extract query parser
test(auth): cover expired tokens
chore: bump django to 5.1
feat(auth)!: require two-factor login -> major  1.2.0 -> 2.0.0
```

1. **type(scope): summary** in the imperative.
2. Tools turn the log into **release notes** and pick the next version.
3. The body explains **why**; the diff already shows what.

STEP BY STEP BRANCHING STRATEGIES

GitHub flow vs Git flow vs trunk-based

	GITHUB FLOW	GIT FLOW	TRUNK-BASED
Long-lived branches	main	main + develop	main
Branch lifetime	days	weeks	hours
Releases	every merge	planned versions	continuous
Unfinished work	on a branch	on a branch	feature flags
Best for	web apps, services	versioned products	strong-CI teams

Most web teams: **GitHub flow**. Say which you'd choose and why.

STEP BY STEP PULL REQUESTS

Protect main

SETTING	EFFECT
Branch protection / rulesets	no direct pushes, no force pushes to main
Required reviews	e.g. 1 approval; stale approvals dismissed on new commits
Required status checks	CI must be green
CODEOWNERS	`/payments/ @shop/payments-team` auto-requested
Require linear history	only squash / rebase merges

COMMON MISTAKE PULL REQUESTS

The 2,000-line pull request

✗ WRONG

```
Title: Update
Description: (empty)
+2,143 -871 in 46 files
"LGTM" after 3 minutes
```

Nobody can review that properly. Bugs slip through, and the branch has drifted for weeks: merge conflicts everywhere.

✓ RIGHT

```
Title: Add coupon codes to checkout
What / Why (SHOP-142) / How I tested
+180 -12 in 5 files, CI green
```

Small PRs, a clear description, green CI. Big features: several PRs behind a feature flag.

IN REAL JOBS TEAMWORK

Your daily team routine

Start a ticket

```
git switch main && git pull
git switch -c feature/SHOP-142-coupons
```

Open the PR

```
git push -u origin feature/SHOP-142-coupons
# GitHub: Compare & pull request
```

Answer the review

```
git commit -m "fix(cart): handle empty coupon"
git push
```

After the merge

```
git switch main && git pull
git branch -d feature/SHOP-142-coupons
```

INTERVIEW SAY THIS IN THE INTERVIEW

“Describe your team’s Git workflow.”

- **GitHub flow**: a short-lived branch per ticket from the latest main.
- Push, open a **PR** with a description; **CI** runs; one or two reviewers approve.
- **Squash-merge** into a **protected** main, which deploys automatically.
- Big features land in small PRs behind **feature flags**.

> **Follow-up they ask next:** ”GitHub flow vs Git flow?” (Git flow adds develop/release/hotfix branches for versioned releases; heavier, more merge pain).

RECAP REMEMBER THESE

Module 7 in 5 lines

1. Branch → PR → CI + review → merge → deploy.
2. Merge buttons: merge commit, **squash**, rebase.
3. **type(scope): summary** commits power changelogs and versions.
4. GitHub flow for most web teams; Git flow for versioned releases.
5. Protect main: reviews, checks, CODEOWNERS.

PRACTICE NOW

Do exercises/ex07_conventional_commits/: three changes, three clean commits.

MODULE

08

Stash, tags & releases

Switch context in seconds, name your releases, and answer ”what changed?”.

YOU WILL LEARN

git stash	01_stash.md
Tags & SemVer	02_tags_and_semver.md
Release notes	03_release_notes.md
Exercise: hotfix release	ex08_hotfix_release/
Interview Q&A	INTERVIEW.md

WHY STASH

Interrupted in the middle of your work

THE PROBLEM

You’re halfway through a feature when the boss says: **”Production is broken, fix it now!”** Your changes aren’t ready to commit, and Git won’t let you switch branches.

THE IDEA

git stash sweeps your unfinished work into a drawer. Fix the bug, come back, and git stash pop it out again.

TRY IT STASH

Put work aside, switch, come back

```
$ git status --short
 M cart.py
?? coupon.py
$ git switch hotfix/rounding
error: Your local changes to the following files would be overwritten by checkout:
        cart.py
Please commit your changes or stash them before you switch branches.
Aborting
$ git stash push -u -m "coupon WIP"
Saved working directory and index state On main: coupon WIP
$ git switch -q hotfix/rounding
$ git switch -q main
$ git stash pop -q
$ git status --short
 M cart.py
?? coupon.py
```

1. Git refuses to switch: your edit would be lost.
2. **-u** includes the new file coupon.py.
3. **pop** restores the work and removes the stash.

STEP BY STEP STASH

Stash commands

COMMAND	DOES
`git stash push -u -m "msg"`	save changes + new files, clean the tree
`git stash list`	stash@{0} (newest), stash@{1}, …
`git stash pop`	re-apply the newest and drop it
`git stash apply stash@{1}`	re-apply one and keep it
`git stash show -p stash@{0}`	see what’s inside
`git stash branch fix-later`	turn a stash into a branch

> Longer than a day? Commit on a branch instead: stashes are easy to forget.

PICTURE IT TAGS

A tag is a bookmark that never moves

🏷️ A **branch** moves forward with every commit. A **tag** stays on one commit forever: perfect for "this is exactly what we released as v1.2.0".

lightweight
git tag v1.0.0: just a name

→

annotated
git tag -a v1.0.0 -m ...: tagger, date, message

→

push
git push origin v1.0.0 (not automatic!)

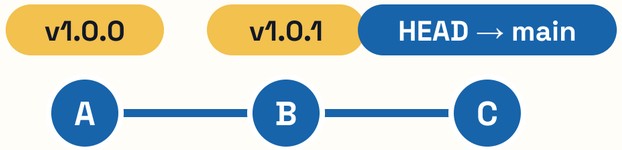
→

release
CI builds + GitHub Release

PICTURE IT TAGS

Tags mark releases

Tags vs branches



1. **v1.0.0** and **v1.0.1** stay where they are forever.
2. **main** moves with every new commit.
3. **git describe**: v1.0.1-1-g<id> = 1 commit after v1.0.1.

TRY IT TAGS

Annotated tags and git describe

```
$ git tag -a v1.0.0 -m "First public release"
$ git cat-file -t v1.0.0
tag
$ echo "wishlist" > wishlist.txt
$ git add wishlist.txt
$ git commit -qm "feat: wishlist"
$ git describe --tags
v1.0.0-1-g2d2ebce
$ git push origin --follow-tags
To ../github/shop.git
   b14578b..2d2ebce  main -> main
 * [new tag]         v1.0.0 -> v1.0.0
```

1. **cat-file -t**: an annotated tag is its own object.
2. **describe**: v1.0.0-1-g<id> = 1 commit after v1.0.0.
3. **--follow-tags** pushes annotated tags along with the commits.

STEP BY STEP SEMVER

Semantic Versioning: MAJOR.MINOR.PATCH

CHANGE	EXAMPLE	BUMP
`bug fix only`	fix: round totals	1.4.2 → **1.4.3**
`new feature, backwards compatible`	feat: wishlist	1.4.2 → **1.5.0**
`breaking change`	feat!: new API format	1.4.2 → **2.0.0**
`pre-release`	release candidate	**2.0.0-rc.1**

> Conventional Commits (Module 7) make the bump automatic.

TRY IT RELEASE NOTES

What's in v1.1.0?

```
$ git log --oneline v1.0.0..v1.1.0
782bd65 feat: wishlist
6b0a5bf fix: rounding in cart
5057df7 feat: coupons
$ git shortlog -sn v1.0.0..v1.1.0
     2  Priya
     1  Vishal
```

1. **A..B** = commits in B that aren't in A.
2. **shortlog -sn**: who contributed.
3. Paste into the GitHub Release notes.

COMMON MISTAKE TAGS

Moving a public tag

✗ WRONG

```
git tag -f v1.2.0 HEAD
git push --force origin v1.2.0
# customers' v1.2.0 != your v1.2.0
```

Anyone who already fetched v1.2.0 keeps the old commit. Two different ”v1.2.0” builds now exist: a support nightmare.

✓ RIGHT

```
# fix it with a NEW version
git tag -a v1.2.1 -m "Fix packaging"
git push origin v1.2.1
```

Released tags are immutable. Mistake? Ship the next patch version.

IN REAL JOBS RELEASES

Releases at work

Cut a release

```
git tag -a v2.3.0 -m "Release 2.3.0"
git push origin v2.3.0
```

CI on tag push

```
on: push: tags: ["v*"]
# build, test, publish, GitHub Release
```

Which version is deployed?

```
git describe --tags   # v2.3.0-4-g1a2b3c
```

Quick context switch

```
git stash -u && git switch hotfix
...
git switch - && git stash pop
```

INTERVIEW SAY THIS IN THE INTERVIEW

“Lightweight vs annotated tags? How do you release?”

- A **lightweight** tag is just a name; an **annotated** tag stores tagger, date and message, and can be signed.
- I use annotated tags with **SemVer**: v2.3.0.
- Tags aren’t pushed automatically: **git push origin v2.3.0** or **--follow-tags**.
- Pushing the tag triggers CI to build, publish and create the GitHub Release.
- Released tags never move; mistakes get a new patch version.

> **Follow-up they ask next:** ”What does git stash do?” (saves uncommitted changes on a stack and cleans the tree; -u for untracked; pop restores).

RECAP REMEMBER THESE

Module 8 in 5 lines

1. **git stash push -u** → switch → **git stash pop**.
2. Tags name releases; use **annotated** tags.
3. Push tags explicitly: **origin v1.2.0** / **--follow-tags**.
4. SemVer: **MAJOR.MINOR.PATCH**.
5. **log A..B** + **shortlog -sn** = release notes.

PRACTICE NOW

Do exercises/ex08_hotfix_release/: stash, tag, restore.

MODULE

09

Git internals

Open the hood: objects, trees, refs and HEAD. After this, Git stops being magic.

YOU WILL LEARN

Objects	`01_objects.md`
Refs & HEAD	`02_refs_and_head.md`
A commit by hand	`03_commit_by_hand.md`
Exercise: plumbing commit	`ex09_commit_by_hand/`
Interview Q&A	`INTERVIEW.md`

WHY INTERNALS

Why look inside .git?

THE PROBLEM

Why does a rebase change commit IDs? Why is creating a branch instant? How can reflog bring back ”deleted” commits? Memorising commands without the model behind them makes Git feel random.

THE IDEA

Git is a small database of objects plus name tags (refs). Learn the 4 object types and everything else follows.

PICTURE IT OBJECTS

Four kinds of objects

📚 A library where every item is filed under a **fingerprint** (hash) of its content: identical content gets the same fingerprint and is stored once. Change one letter and you get a new fingerprint.

`blob`
file CONTENT (no name)

→

`tree`
a folder: names → blobs / trees

→

`commit`
tree + parent(s) + author + message

→

`tag`
annotated name for an object

TRY IT OBJECTS

Walk from a commit down to a blob

```
$ git cat-file -p HEAD
tree 362d89e03cdf686deaa193f9f07649a31e67343b
author Vishal <vishal@example.com> 1793088180 +0000
committer Vishal <vishal@example.com> 1793088180 +0000

First snapshot
$ git cat-file -p HEAD^{tree}
100644 blob ce013625030ba8dba906f756967f9e9ca394464a    a.txt
100644 blob ce013625030ba8dba906f756967f9e9ca394464a    copy-of-
a.txt
040000 tree 5234dadb279379214f5a218b331d5db0a198db4f    src
$ git ls-tree -r HEAD
100644 blob ce013625030ba8dba906f756967f9e9ca394464a    a.txt
100644 blob ce013625030ba8dba906f756967f9e9ca394464a    copy-of-
a.txt
100644 blob 9cf595fed1e892474f91a6d40ebb96d21c66b89d    src/app.py
$ git cat-file -p HEAD:a.txt
hello
```

1. The commit points to ONE tree (the snapshot).
2. Two files, same content: **one blob**.
3. Snapshots, not diffs; unchanged files are reused.

STEP BY STEP HASHES

Why IDs change after rebase and amend

A COMMIT'S ID IS A HASH OF...	CHANGED BY
`its tree (the snapshot)`	any file change
`its parent ID(s)`	**rebase** (new parent)
`author + date`	--reset-author, rebasing (committer date)
`the message`	**amend**, reword

> Change anything → a new hash → a new commit. That's why rewritten history is "different" history.

TRY IT REFS AND HEAD

Branches and HEAD are tiny text files

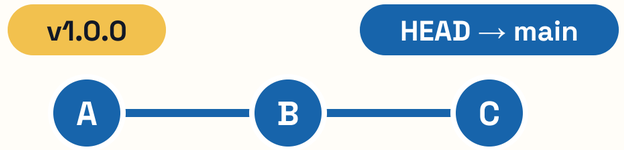
```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
bcf013a339ffca30ac033dc317615cef294a4821
$ git show-ref
bcf013a339ffca30ac033dc317615cef294a4821 refs/heads/main
bcf013a339ffca30ac033dc317615cef294a4821 refs/tags/v1.0.0
$ git switch --detach HEAD~0
HEAD is now at bcf013a Version 1
$ cat .git/HEAD
bcf013a339ffca30ac033dc317615cef294a4821
```

1. A branch = a file with one commit ID.
2. HEAD = ”ref: refs/heads/main” (symbolic).
3. Detached HEAD = HEAD holds a commit ID directly.

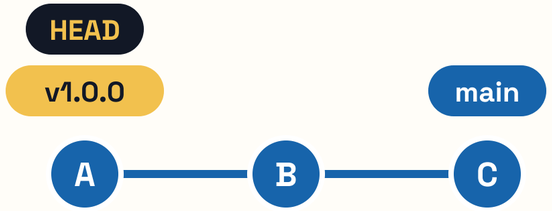
PICTURE IT HEAD

Attached vs detached HEAD

Normal: HEAD → main



`git switch --detach v1.0.0`



Detached: HEAD points at a **commit**, not a branch. New commits here belong to no branch, so create one with **git switch -c**.

STEP BY STEP REFS AND HEAD

Naming commits

EXPRESSION	MEANS
`HEAD~1` = `HEAD~` = `HEAD^`	the (first) parent
`HEAD~3`	three generations back (first parents)
`HEAD^2`	the SECOND parent of a merge commit
`main@{1}`	where main was one move ago (reflog)
`v1.0.0^{tree}`	the tree of the commit a tag points to
`HEAD:src/app.py`	a file's blob in a commit

TRY IT PLUMBING

A commit built by hand

```
$ git hash-object -w hand.txt
1d00f59d814e68d3cbf6f8b74793500b5e916ed0
$ git write-tree
6090a5336cb98585d4646beed4c54b9438588a9f
$ git commit-tree 6090a53 -p HEAD -m "Built by hand"
5d86025e830748472b7b7e575f01623a2c168b02
```

1. **hash-object -w**: store the content → blob.
2. **update-index** + **write-tree**: index → tree.
3. **commit-tree** → commit; **update-ref** moves the branch.

COMMON MISTAKE DETACHED HEAD

Committing in detached HEAD, then leaving

✗ WRONG

```
git switch --detach v1.0.0
# ...two commits of experiments...
git switch main
# warning: you are leaving 2 commits behind
```

Those commits belong to no branch. They're not deleted yet, but they're easy to lose and will be garbage-collected.

✓ RIGHT

```
git switch --detach v1.0.0
# ...commits...
git switch -c experiment     # keep them on a branch
# already left? git reflog -> git branch rescue <id>
```

Create a branch before leaving, or rescue the commits with the reflog.

IN REAL JOBS INTERNALS

Where internals help at work

Scripts and CI

```
git rev-parse --short HEAD   # image tag
git rev-parse --abbrev-ref HEAD
```

Inspect without checking out

```
git show v1.2.0:config/settings.py
git ls-tree -r --name-only v1.2.0
```

Repo size problems

```
git count-objects -vH
git gc
```

Understand ”weird” history

```
git cat-file -p <merge-commit>  # 2 parents
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How does Git store data internally?”

- A content-addressable **object database**: blobs (content), trees (folders), commits, annotated tags.
- Each object is named by the **hash of its content**; identical content is stored once.
- Commits are **snapshots**: a tree + parent(s) + metadata.
- Branches, tags and HEAD are **refs**: tiny files pointing at object IDs.
- That’s why branches are cheap and rebased commits get new IDs.

> **Follow-up they ask next:** ”What is a detached HEAD?” (HEAD points at a commit, not a branch; create a branch to keep new commits).

RECAP REMEMBER THESE

Module 9 in 5 lines

1. 4 objects: **blob**, **tree**, **commit**, **tag**, named by content hash.
2. Commits are snapshots that point to a tree and parent(s).
3. Refs (branches, tags) are files with IDs; HEAD points to a ref.
4. **HEAD~n** first parents, **HEAD^2** second parent.
5. git commit = write-tree + commit-tree + update-ref.

PRACTICE NOW

Do exercises/ex09_commit_by_hand/: a real commit with plumbing only.

MODULE

10

Debugging with history

History is evidence: filter the log, blame a line, search with the pickaxe, and bisect a bug.

YOU WILL LEARN

git log like a detective	01_log_like_a_pro.md
blame & the pickaxe	02_blame_and_pickaxe.md
git bisect	03_bisect.md
Exercise: find the bug	ex10_find_the_bug/
Interview Q&A	INTERVIEW.md

WHY DEBUGGING

”It worked last week”

THE PROBLEM

Prices are wrong in production. 200 commits went in since the last good release. Reading them one by one takes a day, and you don’t even know which file to look at.

THE IDEA

Git history is evidence. Filter it (log), trace a line (blame, -S), and let bisect find the culprit in ~8 steps.

TRY IT GIT LOG

Filter the history

```
$ git log --oneline --author=Priya
4034e86 Coupon: expire after 30 days
3629b31 Add coupon codes
$ git log --oneline -i --grep=coupon
4034e86 Coupon: expire after 30 days
e0d95c2 Document coupons
3629b31 Add coupon codes
$ git log --oneline -- cart.py
766dba7 Fix rounding in cart
e426c38 Add cart
$ git shortlog -sn HEAD
     3  Vishal
     2  Priya
     1  Raj
```

1. By person, by message, by file.
2. Also **--since**, **--until**, **--follow**, **--no-merges**.
3. **shortlog**: commits per author.

TRY IT BLAME AND PICKAXE

Who changed this line? When did it appear?

```
$ git blame -s -L 1,3 cart.py
c2a7a6f3 1) def total(items, discount=0):
b863aded 2)     rate = 1 - min(discount, 0.5)
c2a7a6f3 3)     return sum(items) * rate
$ git log --oneline -S discount
c2a7a6f Add discount
$ git log --oneline -S "min("
b863ade Cap discount at 50%
$ git grep -n rate HEAD~1
HEAD~1:cart.py:2:    rate = 1 - discount
HEAD~1:cart.py:3:    return sum(items) * rate
```

1. **blame**: the last commit per line.
2. **-S** (pickaxe): commits that added/removed the text.
3. **git grep** searches any version.

PICTURE IT BISECT

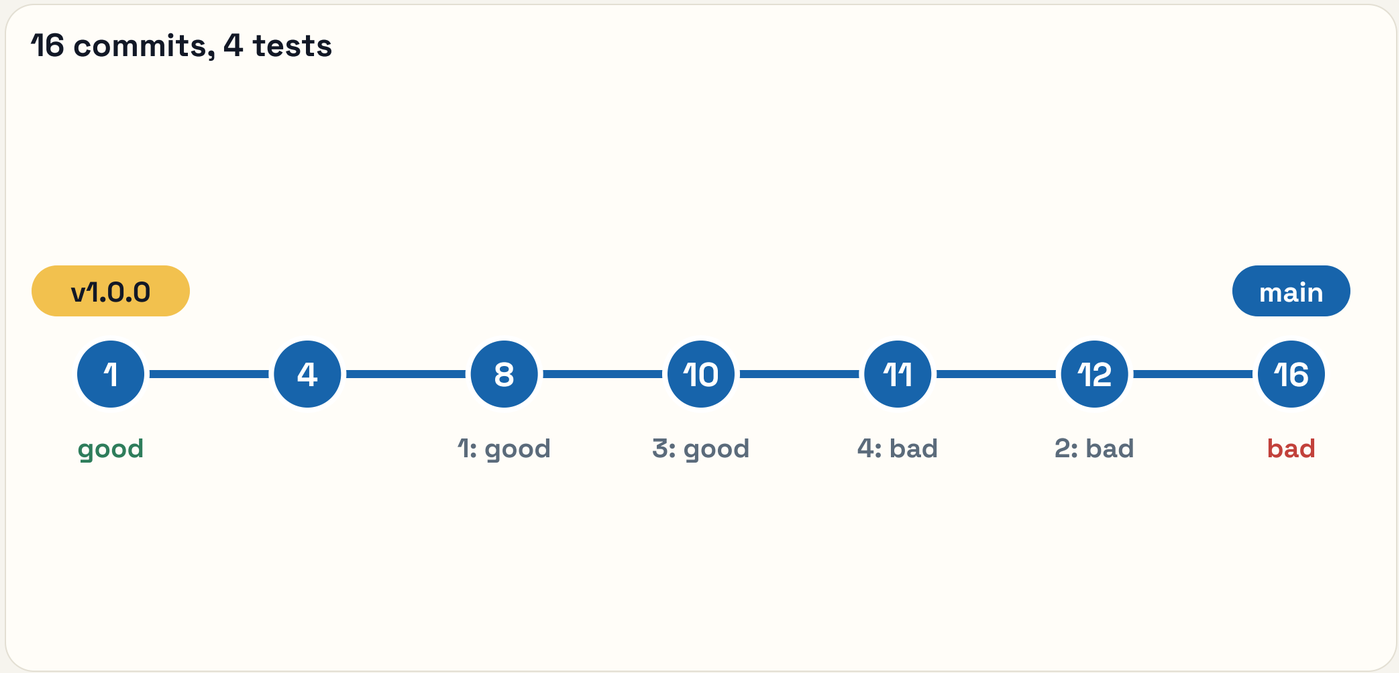
bisect = the number-guessing game

🎯 ”I’m thinking of a number between 1 and 1,000.” ”Higher or lower?” Halving each time finds it in 10 guesses. bisect does the same with commits: **good or bad?**



PICTURE IT BISECT

Binary search through history



1. Test the middle, throw away half.
2. 16 commits → 4 tests; 1,000 → ~10.
3. **11** is the first bad commit.

TRY IT BISECT

Let a script do the testing

```
$ git bisect start HEAD v1.0.0
Bisecting: 7 revisions left to test after this (roughly 3 steps)
[3de9aec6c0e8012786e5cd0044dc2b1e0d22e570] Change 8
$ git bisect run sh check_price.sh
running 'sh' 'check_price.sh'
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[a88ec4c0211c576eb0e32236fa3f1b5e6b03ad25] Change 12
running 'sh' 'check_price.sh'
Bisecting: 1 revision left to test after this (roughly 1 step)
[2059072525fea07f539dcb4609ab43ffb04b776e] Change 10
running 'sh' 'check_price.sh'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[ccd5ec2d091dd68eec1516a4d09f2cf0704d5a80] Change 11
running 'sh' 'check_price.sh'
ccd5ec2d091dd68eec1516a4d09f2cf0704d5a80 is the first 'bad' commit
commit ccd5ec2d091dd68eec1516a4d09f2cf0704d5a80
Author: Vishal <vishal@example.com>
Date:   Tue Oct 27 08:23:00 2026 +0000

    Change 11

 CHANGELOG.txt | 1 +
 price.txt     | 2 +-
 2 files changed, 2 insertions(+), 1 deletion(-)
bisect found first 'bad' commit
```

1. Exit code 0 = good, 1-127 = bad, 125 = skip.
2. 16 commits → 4 checks.
3. Then **git bisect reset** to go back to main.

STEP BY STEP DEBUGGING

Which tool for which question?

QUESTION	TOOL
What changed recently in payments/?	`git log --oneline --since="1 week ago" -- payments/`
Who wrote this odd line, and why?	`git blame -L 40,60 file` → `git show <id>` → the PR
When was this function removed?	`git log -S "def apply_coupon" --oneline`
Which commit broke the build?	`git bisect run pytest -x`
Is this TODO in the released version?	`git grep TODO v2.3.0`

COMMON MISTAKE BISECT

A flaky test leads bisect astray

✗ WRONG

```
git bisect run pytest
# runs the WHOLE suite: unrelated
# flaky tests fail randomly
```

bisect trusts every answer. One wrong "bad" sends it into the wrong half, and it confidently blames an innocent commit.

✓ RIGHT

```
git bisect run pytest tests/test_prices.py -x
# a small, reliable test that
# reproduces exactly this bug
```

Write a tiny, deterministic reproduction script first, then bisect with it.

IN REAL JOBS DEBUGGING

History at work

Production regression

```
git bisect start HEAD v2.3.0
git bisect run ./repro.sh
```

Understand a decision

```
git blame -L 120,130 billing.py
git show a1b2c3d   # + the linked PR
```

Ignore formatting commits in blame

```
echo <reformat-id> >> .git-blame-ignore-revs
```

Release-level history

```
git log --first-parent --oneline main
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you find which commit introduced a bug?”

- **git bisect** between a known good commit (last release tag) and a bad one (HEAD).
- Binary search: about **log2(n)** steps, so 1,000 commits need ~10 checks.
- Best with **git bisect run** and a small script that reproduces the bug.
- Then **git show** the culprit, read its PR, write a test, fix.
- For a single line: **git blame**; for added/removed code: **git log -S**.

> **Follow-up they ask next:** ”A commit in the range doesn’t build?” (git bisect skip, or exit 125 from the script).

RECAP REMEMBER THESE

Module 10 in 5 lines

1. **log** filters: --author, --grep, --since, -- path, --follow.
2. **blame -L**: who last changed these lines.
3. **log -S / -G**: when code appeared or disappeared.
4. **git grep**: search any version.
5. **bisect run**: the first bad commit in ~log2(n) steps.

PRACTICE NOW

Do exercises/ex10_find_the_bug/: 30 commits, one bug, bisect it.

MODULE

11

Advanced Git

Hooks, worktrees, submodules, line endings, secrets in history, LFS and huge repos.

YOU WILL LEARN

Hooks	`01_hooks.md`
Worktrees	`02_worktrees.md`
Submodules & subtree	`03_submodules_and_subtree.md`
Line endings	`04_line_endings_and_attributes.md`
Secrets, LFS, signing	`05_secrets_big_files_and_signing.md`
Exercise: secret guard hook	`ex11_secret_guard_hook/`
Interview Q&A	`INTERVIEW.md`

WHY ADVANCED GIT

Problems that appear in real teams

THE PROBLEM

An API key pushed to GitHub. A diff where all 500 lines ”changed”. A hotfix needed while your dev server is running. A 3 GB repository nobody can clone.

THE IDEA

Each has a precise tool: hooks, .gitattributes, worktrees, filter-repo, LFS, partial clones.

TRY IT HOOKS

A pre-commit hook in 5 lines

```
#!/bin/sh
if git diff --cached | grep -qE "(PASSWORD|SECRET|API_KEY)="; then
  echo "pre-commit: a secret is staged. Commit blocked."
  exit 1
fi
```

1. Lives in **.git/hooks/pre-commit** (make it executable).
2. Exit 1 = the commit is blocked.
3. Others: **commit-msg**, **pre-push**; share with pre-commit.com.

TRY IT HOOKS

The hook in action

```
$ echo "API_KEY=sk_live_123" > settings.py
$ git add settings.py
$ git commit -m "Add settings"
pre-commit: a secret is staged. Commit blocked.
$ git log --oneline
3d365c0 Initial commit
```

1. The commit never happened.
2. Bypass: **--no-verify**, so CI must run the same checks.
3. Real teams: ruff, gitleaks, detect-secrets.

TRY IT WORKTREES

Two branches open at the same time

```
$ git worktree add ../shop-hotfix hotfix/rounding
HEAD is now at dd073e8 Add cart
Preparing worktree (checking out 'hotfix/rounding')
$ git worktree list
~/project     dd073e8 [feature/coupons]
~/shop-hotfix dd073e8 [hotfix/rounding]
$ git status --short
?? coupon.py
```

1. A second folder sharing the same .git.
2. Your unfinished feature stays untouched.
3. **git worktree remove** when done.

STEP BY STEP REUSING CODE

Submodule vs subtree vs package

	SUBMODULE	SUBTREE	PACKAGE (PIP/NPM)
What's in your repo	a pointer to a commit	a copy of the files	a version number
Clone needs	`--recurse-submodules`	nothing special	pip install
Update	`git submodule update --remote`	`git subtree pull`	bump the version
Best for	pinned infra modules, protos	vendoring small libs	most shared libraries

TRY IT LINE ENDINGS

Fix CRLF vs LF once for the whole team

```
$ git ls-files --eol
i/crlf  w/crlf  attr/text=auto eol=lf   deploy.sh
$ cat .gitattributes
* text=auto eol=lf
*.bat text eol=crlf
*.png binary
$ git add --renormalize .
$ git add .gitattributes
$ git commit -qm "Normalise line endings"
$ git ls-files --eol
i/lf    w/lf    attr/text=auto eol=lf   .gitattributes
i/lf    w/crlf  attr/text=auto eol=lf   deploy.sh
```

1. **i/crlf**: the repo stored Windows endings.
2. **.gitattributes** is the team rule.
3. After renormalising: **i/lf**.

COMMON MISTAKE SECRETS

”Fixing” a leaked key with a new commit

✗ WRONG

```
# pushed: API_KEY=sk_live_123
git rm --cached .env
git commit -m "Remove secret"
git push
# the key is still in history (and already scraped)
```

Anyone can read the old commit. Bots find public keys within minutes. The key is compromised.

✓ RIGHT

```
# 1. ROTATE the key now (provider dashboard)
# 2. clean history:
git filter-repo --replace-text expressions.txt
git push --force --all
# 3. prevent: .gitignore, gitleaks, push protection
```

Rotate first. Then clean history and make everyone re-clone.

STEP BY STEP BIG REPOS

Big files and huge repositories

PROBLEM	TOOL
Large binaries (videos, models, PSDs)	**Git LFS**: `git lfs track "*.mp4"`
CI only needs the latest code	`git clone --depth 1` (shallow)
History yes, all blobs no	`git clone --filter=blob:none` (partial)
Only some folders of a monorepo	`git sparse-checkout set services/api`
Prove who made a commit	signed commits (SSH/GPG) → **Verified**

IN REAL JOBS ADVANCED GIT

Senior-level habits

Shared hooks

```
pip install pre-commit
pre-commit install    # .pre-commit-config.yaml
```

Day-one repo hygiene

```
.gitignore + .gitattributes + CODEOWNERS
```

Fast CI checkouts

```
git clone --depth 1 --filter=blob:none <url>
```

Signed commits

```
git config --global gpg.format ssh
git config --global commit.gpgsign true
```

INTERVIEW SAY THIS IN THE INTERVIEW

“You pushed a password to GitHub. What do you do?”

- **Rotate the credential immediately**: assume it’s compromised.
- Not pushed yet? Amend or rebase it out.
- Pushed: **git filter-repo** to remove it from all history, force-push, everyone re-clones.
- Prevent: .gitignore, pre-commit secret scanning, GitHub **push protection**, a vault for secrets.

> **Follow-up they ask next:** ”What are Git hooks?” (scripts on events like pre-commit/commit-msg/pre-push; share via pre-commit; CI repeats them).

RECAP REMEMBER THESE

Module 11 in 5 lines

1. **Hooks** guard commits locally; CI repeats the checks.
2. **worktree**: another branch in another folder, same repo.
3. **Submodules** pin other repos; packages are often simpler.
4. **.gitattributes** + **--renormalize** end line-ending wars.
5. Leaked secret: rotate, filter-repo, prevent. Big files: LFS.

PRACTICE NOW

Do exercises/ex11_secret_guard_hook/ : block secrets with a hook.

MODULE

12

Capstone: rescue a messy repo

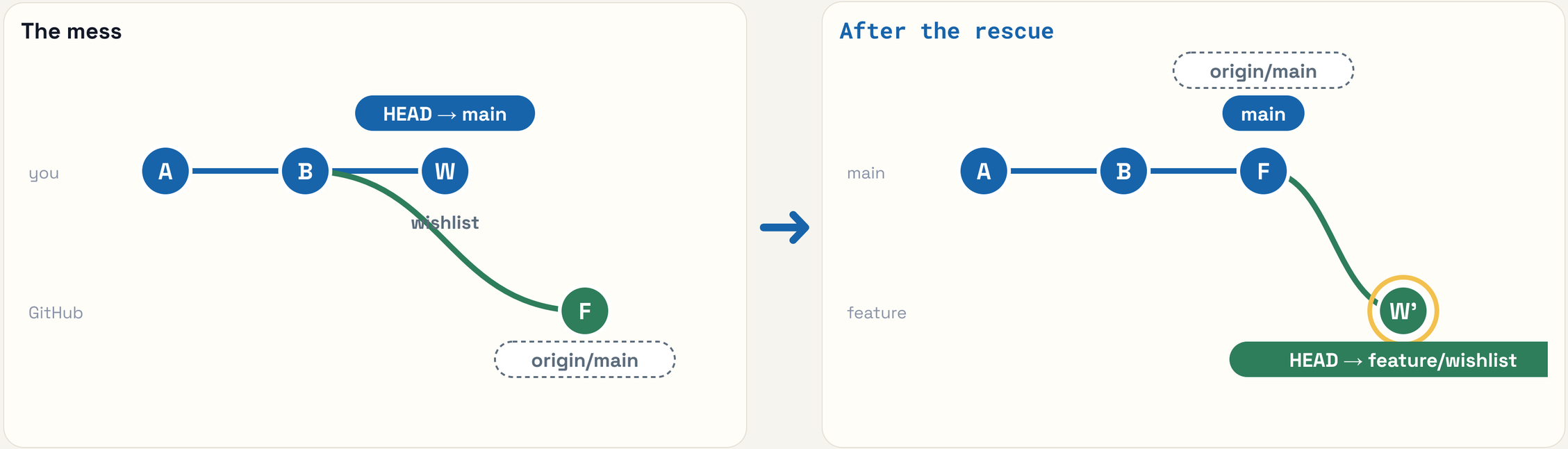
Look, find, plan, act: fix any broken repository calmly and ship a release.

YOU WILL LEARN

Rescue walkthrough	`01_rescue_walkthrough.md`
Exercise: rescue + release	`ex12_rescue_the_repo/`
Final exam	`QUIZ.md`
Interview Q&A	`INTERVIEW.md`

PICTURE IT RESCUE

The rescue, as a picture



main = origin/main again; your work sits on **feature/wishlist**, on top of the latest main.

WHY RESCUE

”Help, I broke Git!”

THE PROBLEM

Committed to main by mistake. Pushed a debug file. A teammate pushed meanwhile. The release has to go out today. Typing random commands from Stack Overflow makes it worse.

THE IDEA

Use the same 4 steps every time: LOOK, FIND, PLAN, ACT. With a safety branch, every step is reversible.

PICTURE IT RESCUE

Look, find, plan, act

🩺 Like a doctor: check the symptoms, read the history, decide on the treatment, then treat one step at a time and check after each one.

LOOK
status -sb, log --graph --all, stash list

→

FIND
reflog, log origin/main..main

→

PLAN
what should each branch look like?

→

ACT
git branch backup, then small steps

TRY IT LOOK AND FIND

Diagnose the mess

```
$ git fetch -q
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git log --oneline --graph --all
* 76eaf96 Add wishlist
| * e6f6181 fix: round totals
|/
* 7cb5715 Initial commit
$ git log --oneline origin/main..main
76eaf96 Add wishlist
```

1. **ahead 1, behind 1**: main diverged from origin.
2. My commit sits on main by mistake.
3. The teammate's fix is on origin/main.

TRY IT ACT

Move the commit and sync with the team

```
$ git branch backup
$ git branch feature/wishlist
$ git reset -q --hard origin/main
$ git switch -q feature/wishlist
$ git rebase -q main
$ git log --oneline --graph --all
* d12811d Add wishlist
* e6f6181 fix: round totals
| * 76eaf96 Add wishlist
|/
* 7cb5715 Initial commit
```

1. **backup** first: free insurance.
2. main = origin/main again.
3. The feature sits on top of the latest main.

STEP BY STEP CHEAT SHEET

"Oh no" → the fix

SITUATION	FIX
Committed to main by mistake	`git branch x; git reset --hard origin/main`
Wrong message / forgot a file	`git commit --amend`
Bad commit already shared	`git revert <id>`
Lost commits	`git reflog` → `git branch rescue <id>`
Push rejected	`git pull --rebase` → push
Need to switch NOW	`git stash -u` → … → `git stash pop`
Which commit broke it?	`git bisect run <test>`
Leaked secret	rotate → `git filter-repo` → prevent

COMMON MISTAKE RESCUE

Panic commands

✗ WRONG

```
rm -rf .git && git init      # "start fresh"
git push --force             # "make it work"
git reset --hard             # with unsaved work
```

These destroy history, teammates' work, or uncommitted changes, and they're the hardest mistakes to undo.

✓ RIGHT

```
git branch backup            # insurance first
git status -sb; git log --graph --all; git reflog
# then one small step at a time
```

Look first. Back up first. Small steps. Almost everything is recoverable if you don't panic.

IN REAL JOBS GIT PRO

You’re ready for real teams

Daily

```
git switch -c feature/x
git add -p && git commit
git pull --rebase && git push
```

Pull requests

```
small PRs, Conventional Commits,
rebase onto main, --force-with-lease
```

Releases

```
git tag -a v1.3.0 -m "Release 1.3.0"
git push origin v1.3.0
```

Emergencies

```
reflog · revert · bisect · stash
```

INTERVIEW SAY THIS IN THE INTERVIEW

“You committed to main instead of a feature branch. Fix it.”

- Keep the work: **git branch feature/x** at the current commit.
- Move main back: **git reset --hard origin/main** (only if not pushed!).
- Continue on **feature/x**; **git rebase main** if main moved.
- Already pushed to shared main? **git revert** instead, and re-apply on a branch.
- Prevent it: protect main on GitHub.

> **Follow-up they ask next:** ”How do you approach a repo in a weird state?” (look, find, plan, act, with a backup branch first).

RECAP REMEMBER THESE

Git Pro in 6 lines

1. Commit small and often; read your diff first.
2. Branch per task; PR + review + CI; protect main.
3. Rebase your own branch; revert shared commits.
4. fetch/pull --rebase; force only with --force-with-lease.
5. reflog, bisect, blame and -S solve the hard problems.
6. Hooks, .gitignore, .gitattributes, secret scanning from day one.

PRACTICE NOW

Do exercises/ex12_rescue_the_repo/, then the final exam and the interview prep.